

Branch Change Report

Complete documentation of all changes, features, and test automation introduced in the current Git branch

TestingCodeImplementation

Base branch:	main
HEAD commit:	e61bf72 – "Add automation for helpPage"
Report generated:	2026-08-12
Prepared by:	Automated repository analysis (Claude Code)

Table of Contents

1. Executive Summary
2. Branch Information
3. Changes Overview
4. Detailed Feature Changes
5. Test Automation Changes
6. File-by-File Changes
7. Git Commit Summary
8. Test Coverage
9. Test Execution Results
10. Known Issues / Limitations
11. Overall Impact
12. Conclusion

1. Executive Summary

This branch introduces a **complete, brand-new automated test suite for the JUNE application's Help menu** — a part of the product that had **zero automated coverage before this branch**. It adds a full Selenium/TestNG page-object layer for the Help icon dropdown, the Feedback form, the Release Notes viewer, and the FAQs page, plus a new TestNG test class (`helpPageTest`) that drives and validates all four areas end to end.

The work exists in two layers on this branch:

- **One committed commit** (`e61bf72` , *"Add automation for helpPage"*) that lays down the initial page objects, data classes, and test class (1,297 lines across 12 new files).
- **Uncommitted work-in-progress in the working tree** (staged new files + unstaged edits) that substantially extends the FAQ coverage added by the commit — wiring up 6 additional FAQ categories (Patent File Wrapper Downloader, AppGen, Oath/Dec & ADS Downloader, OA Shell Draft, Claims Formatter, Support & Help), completing the IDS subcategory coverage, and fixing two real assertion bugs (quote normalization and whitespace normalization) discovered while stabilizing the suite.

Important: the uncommitted portion described in this report is a snapshot of the working tree at report time. If you commit, stage, or discard files afterward, re-run this report to keep it accurate.

By the numbers, the FAQs page automation alone now drives and validates **217 individual FAQ answers** across 10 top-level categories and 16 subcategories, plus 5 "Coming Soon" placeholder states — making it the single largest and most detailed content-verification suite in the framework.

2. Branch Information

Item	Value
Current branch	TestingCodeImplementation
Base branch used for comparison	main
Merge-base commit	abd6209 (identical merge-base with pre-main)
Branch HEAD commit (committed)	e61bf72 – "Add automation for helpPage"
Commits unique to this branch (vs. main)	1
Files added (committed)	12
Files added (staged, not yet committed)	6
Files modified (committed vs. main)	0 — all modifications happened after the commit, in the working tree
Files modified (uncommitted, working tree)	4 (FAQsPage.java, HelpMenuPage.java, IDSFAQData.java, testng.xml) + the 6 staged files (further edited unstaged)
Files deleted / renamed	0
Untracked, out of scope	.mcp.json
Net lines added (committed commit only)	1,297 insertions, 0 deletions across 12 files

3. Changes Overview

At a high level, this branch adds:

- **A new page-object subsystem** under `src/main/java/pages/help/` (`HelpPage` , `HelpMenuPage` , `FeedbackPage` , `ReleaseNotesPage` , `FAQsPage`) modeling every screen reachable from the app's Help icon.
- **A new data layer** under `src/main/java/pages/help/data/` — one class per FAQ category (10 classes) plus `FAQEntry` (a simple question/answer/locator record) and `ReleaseNotesData` (a static map of expected release-note text snippets for 11 releases).
- **One new TestNG test class**, `helpPageTest` , with 4 prioritized `@Test` methods covering the Help menu, Feedback form, Release Notes (including search), and the full FAQs page.
- **Two real bug fixes** inside the FAQ answer-verification logic (quote normalization, whitespace normalization) found while getting the suite to pass reliably.
- **One locator hardening change** to the Help icon button to reduce the chance of matching the wrong element.
- **A `testng.xml` suite reconfiguration** (uncommitted) that currently limits the active suite to only the new `helpPageTest` class, with the previously-active bibliographic-data-extraction and login/logout regression tests commented out.

No changes were made to core framework infrastructure (`BaseTest` , `ConfigReader` , `BasePage` , listeners, retry/report utilities) — this branch is purely additive at the page-object and test layer, reusing the existing `BasePage.waitVisible` / `waitClickable` / `safeClick` / `scrollIntoView` primitives.

4. Detailed Feature Changes

4.1 Help Menu automation (new)

`HelpMenuPage` models the Help icon's dropdown: the June logo icon, the Help button itself, and the four menu items (Feedback, Release Notes, FAQ, Tutorial). `HelpPage` acts as a thin facade composing `HelpMenuPage` with `FeedbackPage`, `ReleaseNotesPage`, and `FAQsPage`, giving test code a single entry point (e.g. `helpPage.clickAndVerifyFAQs()`) instead of wiring four page objects together in every test.

`checkElementsOfHelpIcon()` asserts all four menu items (Feedback, Release Notes, FAQ, Tutorial) are visible after opening the dropdown, and captures a `helpMenuElements` screenshot — the first automated confirmation that the Help dropdown is fully populated.

4.2 Feedback form automation (new)

`FeedbackPage.verifyAndClose()` opens the Feedback option, asserts the "Feedback Form" title and "Rate us" text are visible, then clicks Cancel to close the dialog — a smoke check that the feedback entry point renders correctly and can be dismissed without side effects (no actual feedback is submitted).

4.3 Release Notes automation (new)

`ReleaseNotesPage` is the most complex new page object (228 lines). It does two passes over the Release Notes screen:

- `verifyAllReleases()` — iterates every release listed in the sidebar (11 releases currently: Release 1.0 through 11.0), opens each one, and validates: the detail title matches the sidebar label, a "Key Updates and Enhancements" heading is present, the body content is non-trivial (>50 characters), an introductory sentence is present, at least one bullet point is visible, and — where `ReleaseNotesData` has an entry for that release — every expected text snippet for that release actually appears in the rendered page.
- `verifySearch()` — for every release title, types it into the search box, asserts the filtered list still contains that release, opens it, and re-validates the detail title and heading; then clears the search and asserts the full release count is restored.

`ReleaseNotesData` is a static `LinkedHashMap` of 11 releases (`Release 1.0` – `Release 11.0`), each mapped to an array of expected phrases pulled from the real release-note copy (e.g., Release 11.0's "Patent Image File Wrapper Downloader", Release 10.0's "Global Patent Bibliographic Search"). This ties the automated suite directly to actual shipped product copy, so a future release-notes edit that drops or garbles content will fail the corresponding assertion.

4.4 FAQs page automation (new, and the largest change)

`FAQsPage` (439 lines currently) is the largest new class. `verifyAll()` walks:

1. The page title and back button.

2. All 10 sidebar categories (Getting Started, Bibliographic Data Extraction, Document Generation, IDS, Patent File Wrapper Downloader, AppGen, Oath/Dec & ADS Downloader, OA Shell Draft, Claims Formatter, Support & Help).
3. The two "Beta" badges (Patent File Wrapper Downloader, Claims Formatter).
4. Every category and subcategory's individual FAQ accordions.

For subcategories that have no published FAQ content yet, `runComingSoon()` opens the subcategory and asserts the "This will be available shortly" placeholder is shown instead — deliberately covering the "not yet documented" state rather than skipping it.

`verifyAccordion()` is the core per-question check: it expands the accordion (clicking only if not already expanded), asserts `aria-expanded="true"`, reads the rendered answer text (normalizing non-breaking spaces), asserts a minimum length, then compares a **quote-and-whitespace-normalized** version of the live answer against a normalized version of the expected snippet using **substring containment** (not exact match), and captures a per-question screenshot.

4.5 FAQ data authored this branch

Ten data classes hold the expected question/answer pairs, one per FAQ category. Combined, they define **217 FAQ entries**:

Category	FAQ entries
AppGen	38
IDS (6 populated subcategories)	48
Bibliographic Data Extraction (7 subcategories)	43
Document Generation (6 populated subcategories)	35
Getting Started	12
Claims Formatter	12
OA Shell Draft	11
Patent File Wrapper Downloader	11
Oath/Dec & ADS Downloader	5
Support & Help	2
Total	217

Plus 5 subcategories intentionally left empty as "Coming Soon" placeholders (`IDSFAQData.CORRESPONDING_REF_CHECK`, `DocumentGenerationFAQData.POA_REVOCATION`, `.ATTORNEY_WITHDRAWAL`, `.CORRECT_APPLICANT_NAME`, and an apparently-unused `.CFR`).

Several entries intentionally build their locator with the `FAQEntry(By, String, String)` constructor instead of the convenience `FAQEntry(String xpathContainsText, ...)` constructor, specifically to avoid XPath string-literal conflicts when the question text itself contains an apostrophe or a double quote

(e.g. `"Q: What is \"SB-08 Generator\" in JUNE ?"`, `"Q: What is the 'US Trademark' feature..."`). This is a deliberate, correctly-applied workaround, not an inconsistency.

4.6 Bug fixes made while stabilizing the suite

- **Quote normalization fix** — `FAQsPage.normalizeQuotes()` now correctly maps typographic quotes (`' ' "`) to their straight ASCII equivalents via proper Unicode character literals, so answers rendered with "smart quotes" in the UI can still match hardcoded straight-quote expected strings.
- **Whitespace normalization fix** — a new `normalizeWhitespace()` helper collapses runs of whitespace and trims both the actual and expected text before the `contains()` assertion, so incidental line breaks/spacing differences in the rendered DOM no longer cause false failures.

4.7 Locator hardening

`HelpMenuPage.helpIcon` was tightened from `//button[normalize-space()='Help']` to `//button[.//*[data-testid='HelpOutlineIcon'] and normalize-space()='Help']`, requiring the button to both contain the specific `HelpOutlineIcon` test-id **and** have the text "Help" — reducing the risk of accidentally matching an unrelated button elsewhere in the DOM with the same visible text.

5. Test Automation Changes

5.1 New test class: `src/test/java/test/helpPageTest.java`

Extends `base.BaseTest`. A `@BeforeClass` method logs in via `LoginPage` using credentials from the environment config (`config.getProperty("username"/"password")`), waits for the page to finish loading, and constructs a `HelpPage`.

#	Test method	Pri.	Validates	Type
1	<code>testHelpPage</code>	1	Help dropdown opens and shows all 4 menu options (Feedback, Release Notes, FAQ, Tutorial)	Positive / UI presence
2	<code>testFeedbackPage</code>	2	Feedback form opens with correct title/text and can be cancelled	Positive / UI + interaction
3	<code>testReleaseNote</code>	3	Every release note opens with correct title, heading, content length, intro text, bullets, and expected text snippets; search filters correctly and restores the full list on clear	Positive / content + functional
4	<code>testFAQsPage</code>	4	All 10 FAQ categories, 16 subcategories, 217 individual FAQ answers, 2 Beta badges, and 5 "Coming Soon" placeholders render with correct content	Positive / content regression

Test case detail: `testHelpPage`

- **Steps:** click June icon → click Help icon → assert all 4 dropdown items visible → screenshot.
- **Test data:** none (structural check only).
- **Expected result:** Feedback, Release Notes, FAQ, Tutorial items all visible.
- **Key locators:** `HelpMenuPage.helpIcon`, `.feedbackOption`, `.releaseNotesOption`, `.faqsOption`, `.tutorialOption`.
- **Validation type:** Positive.

Test case detail: `testFeedbackPage`

- **Steps:** click Help icon → click Feedback → assert form title + "Rate us" text visible → click Cancel.
- **Test data:** none.
- **Expected result:** Feedback dialog renders and closes cleanly.
- **Key locators:** `FeedbackPage.feedbackFormTitle`, `.rateUsText`, `.cancelButton`.
- **Validation type:** Positive.

Test case detail: `testReleaseNote`

- **Steps (part 1 — verifyAllReleases):** click Help icon → click Release Notes → for each of 11 releases: click it → assert title match, "Key Updates and Enhancements" heading, content length > 50 chars, intro phrase present, ≥1 bullet visible, and (where mapped) every expected snippet from `ReleaseNotesData` is present → screenshot → back.

- **Steps (part 2 — verifySearch):** re-open Help icon → Release Notes → for each release title: type it into search → assert it's the only/expected match and its detail page opens correctly → clear search → assert full release count is restored.
- **Test data:** `ReleaseNotesData.CONTENT` — 11 releases, each with 7–13 expected text snippets sourced from real release-note copy.
- **Expected result:** all 11 releases open, validate, and are individually searchable.
- **Key locators/methods:** `ReleaseNotesPage.releaseItems` , `.releaseDetailTitle` , `.keyUpdatesHeading` , `.featureBullets` , `.searchField` ; `ReleaseNotesData.hasContent()` / `.getTexts()` .
- **Validation type:** Positive / content-accuracy.

Test case detail: testFAQsPage

- **Steps:** click Help icon → click FAQs → `FAQsPage.verifyAll()` : title → 10 sidebar categories → 2 Beta badges → walk every category/subcategory's FAQ accordions (expand, assert `aria-expanded` , read answer, normalize, assert expected substring present, screenshot) → for unpublished subcategories, assert the "Coming Soon" placeholder instead.
- **Test data:** 217 `FAQEntry` records across the 10 data classes listed in §4.5, plus 5 "Coming Soon" subcategories.
- **Expected result:** every FAQ answer contains its documented expected text; every "Coming Soon" subcategory shows the placeholder message.
- **Key locators/methods:** `FAQsPage.verifyAccordion()` , `.runSubcategoryFAQs()` , `.runComingSoon()` , `.normalizeQuotes()` , `.normalizeWhitespace()` .
- **Validation type:** Positive / content regression (see §10 for the substring-vs-full-text coverage caveat).

5.2 What this suite does not cover

No new API tests, database tests, or negative/invalid-input scenarios were added in this branch. This is a **read-only, content-verification** suite: it opens screens, reads rendered text, and checks it against expected values. It does not submit the Feedback form with data, does not test error/validation messages, and does not exercise any backend or API layer directly.

6. File-by-File Changes

6.1 Committed in e61bf72

File	Lines	Change
<code>FAQsPage.java</code>	+351	New page object; initial version covering title, sidebar categories, beta badges, Getting Started, Bibliographic, Document Generation, and 1 of 7 IDS subcategories.
<code>FeedbackPage.java</code>	+25	New page object for the Feedback dialog.
<code>HelpMenuPage.java</code>	+61	New page object for the Help dropdown (June icon, Help icon, 4 menu items).
<code>HelpPage.java</code>	+52	New facade composing the four Help-related page objects.
<code>ReleaseNotesPage.java</code>	+228	New page object for Release Notes list, detail view, and search.
<code>data/BibliographicFAQData.java</code>	+157	New data class — 43 FAQ entries across 7 subcategories.
<code>data/DocumentGenerationFAQData.java</code>	+141	New data class — 35 FAQ entries across 6 populated subcategories + 4 "Coming Soon" stubs.
<code>data/FAQEntry.java</code>	+37	New simple record class (locator, question, expected answer) with two constructors.
<code>data/GettingStartedFAQData.java</code>	+48	New data class — 12 FAQ entries.
<code>data/IDSFAQData.java</code>	+4	New data class — initial stub (1 of 7 subcategories populated: DOWNLOADER_1449_892).
<code>data/ReleaseNotesData.java</code>	+139	New data class — expected text snippets for 11 releases.
<code>test/helpPageTest.java</code>	+54	New TestNG test class, 4 test methods (see §5.1).

6.2 Uncommitted — staged (new files, currently stubs on disk in the index)

File	Purpose
<code>data/AppGenFAQData.java</code>	New data class for the AppGen FAQ category.
<code>data/ClaimsFormatterFAQData.java</code>	New data class for the Claims Formatter FAQ category.
<code>data/OAShellDraftFAQData.java</code>	New data class for the OA Shell Draft FAQ category.
<code>data/OathDecAdsFAQData.java</code>	New data class for the Oath/Dec & ADS Downloader FAQ category.
<code>data/PatentFileWrapperDownloaderData.java</code>	New data class for the Patent File Wrapper Downloader FAQ category.
<code>data/SupportHelpFAQData.java</code>	New data class for the Support & Help FAQ category.

6.3 Uncommitted — unstaged modifications (working tree, not yet staged/committed)

File	Change	Purpose/Impact
<code>FAQsPage.java</code>	+97 / -8	Wires up 6 new <code>verify*FAQs()</code> methods (Patent File Wrapper Downloader, AppGen, Oath/Dec & ADS, OA Shell Draft, Claims Formatter, Support & Help) into <code>verifyAll()</code> ; uncomments and wires the remaining 4 IDS subcategories; adds <code>normalizeWhitespace()</code> and applies it alongside the quote-normalization fix in <code>verifyAccordion()</code> .
<code>HelpMenuPage.java</code>	+4 / -2	Hardens the <code>helpIcon</code> locator (see §4.7); minor formatting.
<code>data/AppGenFAQData.java</code>	+135 / -2	Stub → full 38-entry FAQ list.
<code>data/ClaimsFormatterFAQData.java</code>	+45 / -1	Stub → full 12-entry FAQ list.
<code>data/OAShellDraftFAQData.java</code>	+39 / -1	Stub → full 11-entry FAQ list.
<code>data/OathDecAdsFAQData.java</code>	+24 / -1	Stub → full 5-entry FAQ list.
<code>data/PatentFileWrapperDownloaderData.java</code>	+59 / -2	Stub → full 11-entry FAQ list.
<code>data/SupportHelpFAQData.java</code>	+11 / -1	Stub → full 2-entry FAQ list.
<code>data/IDSFAQData.java</code>	+194 / -1	Extends the committed 1-subcategory stub to all 6 populated IDS subcategories (SB08_GENERATOR, REMOVE_EMBEDDED_FONTS, REFERENCE_EXTRACTOR, REFERENCE_DOWNLOADER, REFERENCE_COUNT added; CORRESPONDING_REF_CHECK intentionally left as a "Coming Soon" stub).
<code>testng.xml</code>	+10 / -9	Suite reconfigured to run only <code>test.helpPageTest</code> ; the 9 previously-active classes (LoginTest, 7 bibliographic-data-extraction scenario tests, LogoutTest) are commented out. See §10 — this disables most of the regression suite if merged as-is.

7. Git Commit Summary

Commit	Message	Files affected	Purpose	Functional impact
e61bf72	Add automation for helpPage	12 files, +1,297/-0	Introduces the entire Help-menu page-object layer, initial FAQ/Release-Notes data, and the new helpPageTest class	Adds the framework's first automated coverage of the Help icon, Feedback form, Release Notes, and a partial FAQs page (Getting Started, Bibliographic, Document Generation, and 1 IDS subcategory)

Only one commit is unique to this branch relative to `main`. All further FAQ-category expansion, the two bug fixes, the locator hardening, and the `testng.xml` suite change described throughout this report are **uncommitted** in the working tree at report time (see §6.2/6.3).

8. Test Coverage

→ [verifyAllReleases](#) → [verifySearch](#)

Test/Class	Scenario	Type	Expected Result	Status
testHelpPage	Help dropdown shows Feedback/Release Notes/FAQ/Tutorial	Positive	All 4 items visible	See §9
testFeedbackPage	Feedback form opens and cancels	Positive	Title + "Rate us" visible, dialog closes	See §9
testReleaseNote	All 11 releases open with correct title/heading/content/snippets	Positive / content	All assertions pass for 11 releases	See §9
testReleaseNote	Search filters to the correct release; clearing restores full list	Positive / functional	Search + clear behave correctly for 11 terms	See §9
testFAQsPage	Getting Started — 12 FAQ answers	Positive / content	12/12 pass	See §9
testFAQsPage	Bibliographic Data Extraction — 43 FAQ answers / 7 subcategories	Positive / content	43/43 pass	See §9
testFAQsPage	Document Generation — 35 FAQ answers / 6 subcategories + 2 "Coming Soon"	Positive / content + placeholder	35/35 + 2/2 pass	See §9
testFAQsPage	IDS — 48 FAQ answers / 6 subcategories + 1 "Coming Soon"	Positive / content + placeholder	48/48 + 1/1 pass	See §9
testFAQsPage	Patent File Wrapper Downloader — 11 FAQ answers	Positive / content	11/11 pass	See §9
testFAQsPage	AppGen — 38 FAQ answers	Positive / content	38/38 pass	See §9
testFAQsPage	Oath/Dec & ADS Downloader — 5 FAQ answers	Positive / content	5/5 pass	See §9
testFAQsPage	OA Shell Draft — 11 FAQ answers	Positive / content	11/11 pass	See §9
testFAQsPage	Claims Formatter — 12 FAQ answers	Positive / content	12/12 pass	See §9
testFAQsPage	Support & Help — 2 FAQ answers	Positive / content	2/2 pass	See §9

Execution status: see §9 for what was and was not actually verified by running the suite.

9. Test Execution Results

Two distinct activities were performed for this report, and they are reported separately so as not to conflate "compiles" with "passes":

9.1 Compilation check (performed for this report — safe, no side effects)

```
mvn -o compile          → BUILD SUCCESS, 0 errors
mvn -o test-compile      → BUILD SUCCESS, 0 errors
```

The full working tree — including all uncommitted changes described in §6.2/6.3 — compiles cleanly with no errors or warnings.

9.2 Full suite execution (mvn test)

Not run for this report. Running the suite launches a real Chrome browser, logs into a live application environment with real stored credentials, and (on completion) sends a real report email via `EmailUtil` (SMTP to a live account, with a fixed recipient). Given those side effects, it was not run automatically as part of generating this report. If a fresh execution run is wanted, confirm the target environment (qa/uat/prod — note prod points at a live production login) explicitly first.

9.3 Most recent known execution (from earlier in this working session, not re-verified for this report)

A full run was executed earlier in this session (`ExecutionResults/Run_20260812_110907` , TestNG suite `testng.xml` , class `test.helpPageTest` , browser: Chrome, timestamps 11:09–11:14 on 2026-08-12) against the working tree in roughly its current state. Results, taken directly from that run's console output:

Test method	Result	Notes
<code>testHelpPage</code>	FAILED	TimeoutException waiting for <code>//li[contains(text(),'Feedback')]</code> after clicking the Help icon. Diagnosed earlier this session as a likely toggle-state issue in <code>HelpMenuPage.clickHelpIcon()</code> — unrelated to the FAQ/Release-Notes content work.
<code>testFeedbackPage</code>	FAILED	Same symptom: TimeoutException waiting for <code>//h5[contains(text(),'Feedback Form')]</code> .
<code>testReleaseNote</code>	FAILED	Same symptom: TimeoutException waiting for <code>//li[contains(text(),'Release Notes')]</code> .
<code>testFAQsPage</code>	PASSED	All 10 categories, 16 subcategories, and 217 individual FAQ answers verified successfully, including the newly-wired AppGen (38), Oath/Dec & ADS Downloader (5), OA Shell Draft (11), Claims Formatter (12), and Support & Help (2) categories. All 5 "Coming Soon" placeholders verified correctly.

This is real, previously-observed evidence, not a claim of success invented for this report — but it reflects a specific point in time and has **not been re-run or re-confirmed** as part of producing this document. Treat §9.3 as historical context, and §9.1 as the only result independently verified while writing this report.

10. Known Issues / Limitations

All items below are substantiated directly from the code or from the execution evidence in §9.

1. **testng.xml currently disables most of the regression suite.** The uncommitted working-tree version of `testng.xml` runs only `test.helpPageTest` ; `LoginTest` and all 7 bibliographic-data-extraction scenario tests plus `LogoutTest` are commented out. If this file is committed/merged as-is, CI or scheduled runs would silently stop covering the rest of the application.
2. **HelpMenuPage.clickHelpIcon() has no state verification.** It calls `safeClick(helpIcon)` on the same toggle button every time it's invoked, with no check that the dropdown actually opened (or was already open/closed) before the caller proceeds to wait on a menu item. This was diagnosed as the likely root cause of the 3 real timeout failures in §9.3. The locator hardening added this branch (§4.7) narrows *what* gets clicked but does not add a guard for *state*, so the underlying flakiness risk remains.
3. **Hardcoded Thread.sleep() calls** in `FAQsPage.java` (lines 375, 431) and `ReleaseNotesPage.java` (lines 163, 175) wait a fixed duration for UI animations/search-debounce instead of an explicit condition — a common source of flakiness on slower machines or CI runners.
4. **Substring-only content assertions.** `FAQsPage.verifyAccordion()` asserts the expected snippet is contained in the live answer, not that the full answer text is correct or complete. A regression that corrupts or drops content outside the anchored snippet (but leaves the snippet itself intact) would not be caught by this suite.
5. **Dead code:** `DocumentGenerationFAQData.CFR` (an empty `FAQEntry[]`) is never referenced anywhere in the codebase — `CFR_37` is the array actually used by `FAQsPage.verifyDocumentGenerationFAQs()` . Likely a leftover from renaming.
6. **No negative-path coverage.** This suite only validates that documented content renders correctly; it does not test invalid input, error/validation messages, or failure states for the Feedback form, search, or FAQ page.
7. **Tightly-coupled hardcoded expected text.** 217 FAQ answer snippets and 11 releases' worth of text snippets are hardcoded directly in Java data classes. Any product-copy edit requires a matching, manual update to these files — there is no single source of truth linking the two.
8. **Environment/credential dependency.** Every test in this suite depends on ConfigReader-loaded credentials and a reachable live environment; the suite cannot run offline or against mocked data. (Full detail on plaintext credential storage was covered in an earlier security audit of this repository, and is not re-litigated here.)
9. **Minor style nit:** `FAQsPage.java` currently has no trailing newline at end of file in the working tree.

11. Overall Impact

Before this branch, the Help menu, Feedback form, Release Notes, and FAQs page had **no automated coverage at all** — any regression in that area of the product would only surface through manual testing. This branch closes that gap almost entirely:

- **Structural coverage** of the Help dropdown, Feedback dialog, and Release Notes navigation/search.
- **Deep content-regression coverage** of the FAQs page — 217 individual answers checked against documented expected text, spanning every product feature area the app currently documents (Bibliographic Data Extraction, Document Generation, IDS, Patent File Wrapper Downloader, AppGen, Oath/Dec & ADS Downloader, OA Shell Draft, Claims Formatter, and general Support/Getting-Started content).
- **Release-notes accuracy coverage** — 11 releases' worth of expected copy checked against the live rendered page, which will catch accidental content loss or corruption in that section going forward.
- Two genuine bugs in the verification logic itself (quote handling, whitespace handling) were found and fixed while building this suite, meaning the assertions are now more reliable than a naive first pass would have produced.

The main gap preventing this from being "done" is operational rather than functional: the `testng.xml` suite scoping needs to be reverted (or intentionally re-scoped) before merge so the rest of the regression suite isn't silently dropped, and the `HelpMenuPage.clickHelpIcon()` toggle-state issue should be fixed so the 3 currently-flaky tests stabilize.

12. Conclusion

This branch delivers a substantial, well-structured new automated test suite for the JUNE Help menu and FAQs page — the framework's first coverage of that area, growing to 217 verified FAQ answers and 11 verified release notes by the time the uncommitted work is included. The code compiles cleanly with zero errors. Of the four new test methods, one (`testFAQsPage` , the largest and most detailed) passed completely in its most recent run; the other three failed for a single, well-understood, unrelated reason (a toggle-state gap in the Help-icon click helper) rather than any issue with the FAQ/content work itself. Recommended next steps before merge:

1. Fix `HelpMenuPage.clickHelpIcon()` to verify the dropdown's open state.
2. Restore or intentionally re-scope `testng.xml` so the rest of the regression suite isn't disabled.
3. Commit the substantial uncommitted FAQ-data and `FAQsPage` work described throughout this report, since it currently only exists in the working tree.

Report generated by inspecting the actual source code, git history, and most recent execution artifacts of the
TestingCodeImplementation branch — not from commit messages alone.